

Comparing Algorithms in Asynchronous RL

Sathya Gnanakumar • Daniel Kiely • Ishaan Kalra • Parthav Poudel

Introduction: Why Asynchronous RL?



Traditional Synchronous RL

Most RL performs on-policy updates. The model generating rollouts must be the same one receiving gradient updates.



The Idle Resource Problem

Waiting for long rollouts to finish before training leaves expensive GPUs sitting idle, bottlenecking efficiency.



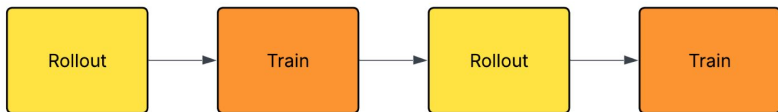
Asynchronous RL (AReaL): Boosting Throughput

- Batching rollouts allows workers to run continuously and independently.
- Trade-off: Training stability is challenged as off-policy updates introduce "staleness".

Motivation: Algorithm Choice in Async RL

Synchronous RL: Fresh data, stop-and-go compute

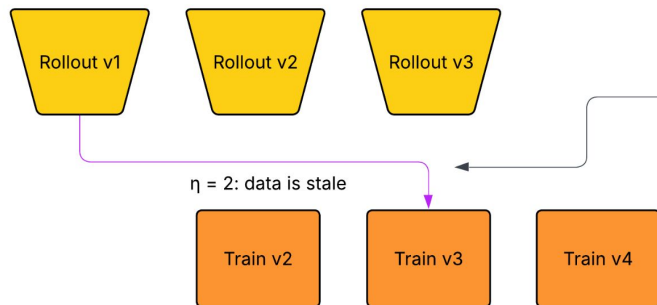
$\eta = 0$: train on freshly generated data



Asynchronous RL: Overlapped compute, but stale data risk

Rollout = model generates answers

Rollout Workers

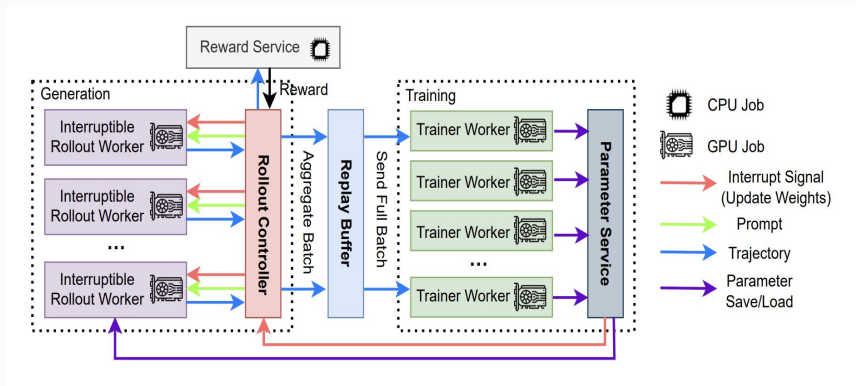


Answer generated by model v1 is used to train model v3

This gap is staleness: data from an older policy is consumed by a newer learner.

What is AReal?

Asynchronous RL system for LLMs with controlled staleness



RL Training System for LLMs

Runs large-scale RL for reasoning models (e.g., GSM8K): rollout generation, reward computation, & policy updates

Fully Async Architecture

Decouples rollout workers and training workers so both run continuously without waiting

Explicit Staleness Control

Allows training on data from older policy versions, with a staleness parameter (η)

Algorithm-System Co-Design

Introduces modified PPO (decoupled objective) to remain stable under async, stale data training

Research Platform for RL Algorithms

Provides a controlled setup to compare algorithms (e.g. GRPO, SAPO, M2PO) with varying staleness

Related Work



Asynchronous RLHF

Shows async training improves efficiency but introduces off-policy issues

(Noukhovitch et al. 2025)



Prosperity before Collapse

How Far Can Off-Policy RL Reach with Stale Data on LLMs?

(Zheng et al. 2025)



MiniMax-M1

Scaling Test-Time Compute Efficiently with Lightning Attention

(Chen et al. 2025)



Stable Asynchrony

Why async RL breaks & high variance from importance sampling ratios

(Huang et al. 2026)



StaleFlow

System-level staleness control + rollout coordination

(Li et al. 2026)

CISPO



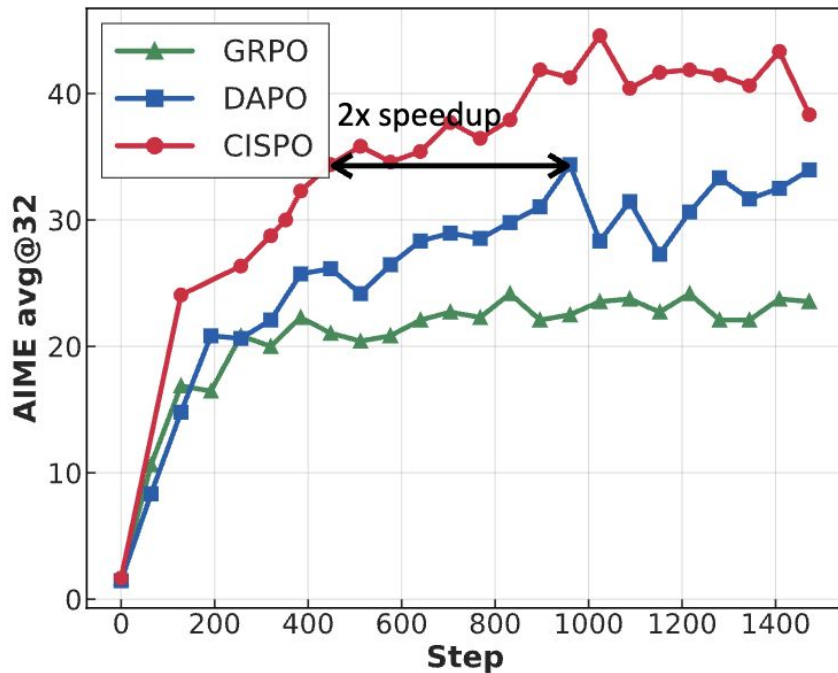
Insight

Tokens associated with reflective behavior ("However", "Wait", etc.) are assigned low probabilities and clipped under vanilla PPO/GRPO.



Solution

Instead of clipping token updates, instead clip importance sampling weights.



MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention

CISPO - Connection to Asynchronous RL



Mechanism of Staleness

Importance sampling ratios grow with staleness.



GRPO vs. CISPO

GRPO zeros out high-ratio tokens, while CISPO ensures each token receives a gradient update.



Empirical Investigation

We investigate whether this clipping strategy is beneficial under increasing staleness levels.



Performance Benchmark

DAPO uses higher clipping bounds, but MiniMax-M1 authors found CISPO performs better.

Our Contribution



Implementation in AReal System

Successfully integrated CISPO into the open-source AReal framework.

- AReal repo: 5k+ stars and 100+ contributors
- First-ever implementation of CISPO in this popular RL library



SOTA Algorithm Comparison

Evaluating how CISPO and other SOTA RL algorithms manage moderate staleness within a unified system architecture.



Real-System Staleness Study

Moving beyond the theoretical simulations of the M2PO paper to analyze staleness effects in a production-grade RL environment.

Experimental Setup



Environment & Model

- Cluster: Zaratan HPC, 2×A100 GPUs
- Model: Qwen2.5-1.5B-Instruct
- Dataset: GSM8K math reasoning



Training Configuration

- Epochs: 10
- Conditions: 12 unique variants
- Metrics: Final task reward



Algorithms Compared

Core Baselines & Proposals

- GRPO:** Proximal Policy Optimization
- CISPO:** Clip-based Importance Sampling
- SAPO:** Staleness-Aware Optimization
- M2PO:** Multi-Objective Policy Optimization



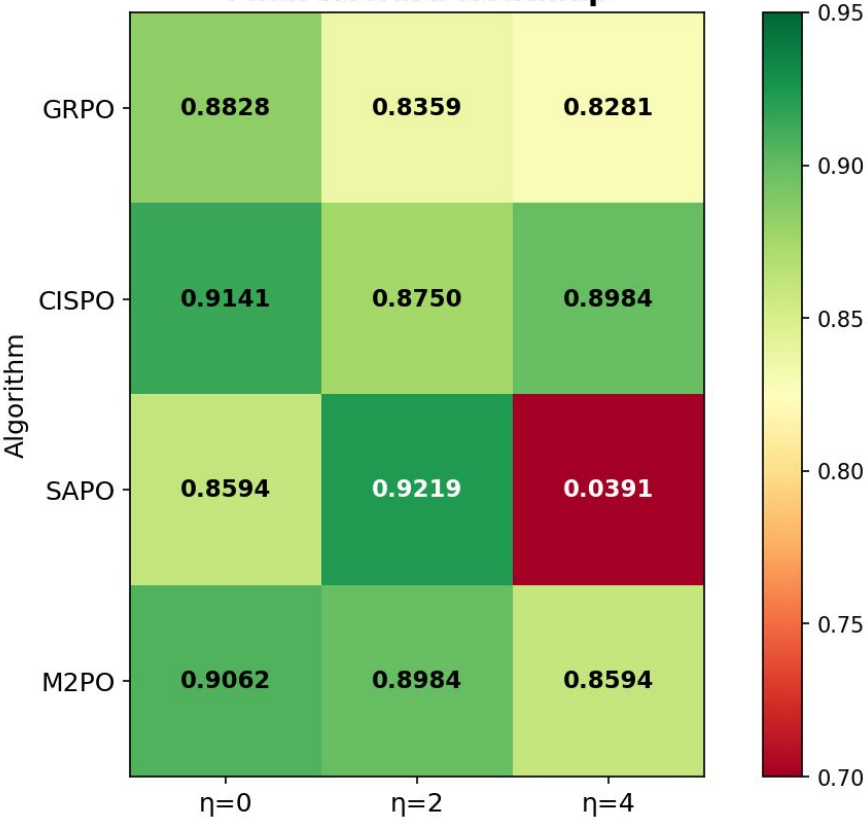
Staleness Levels (η)

Asynchronous Impact Study

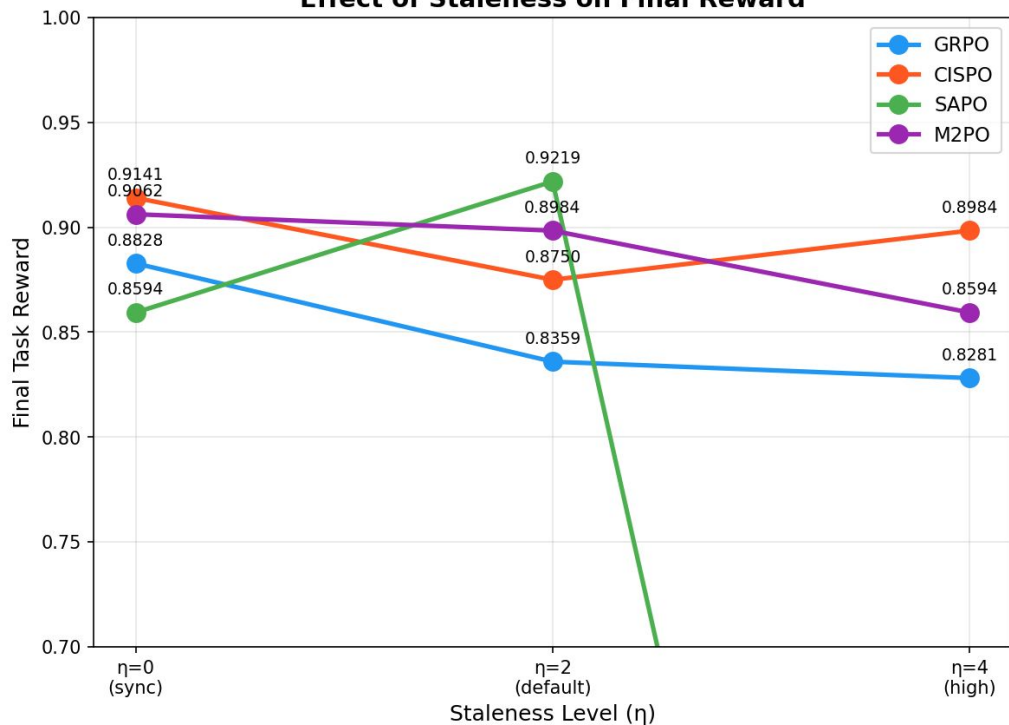
- $\eta=0$ Synchronous (No staleness)
- $\eta=2$ Moderate staleness (Default)
- $\eta=4$ High staleness levels

Performance Analysis Plots

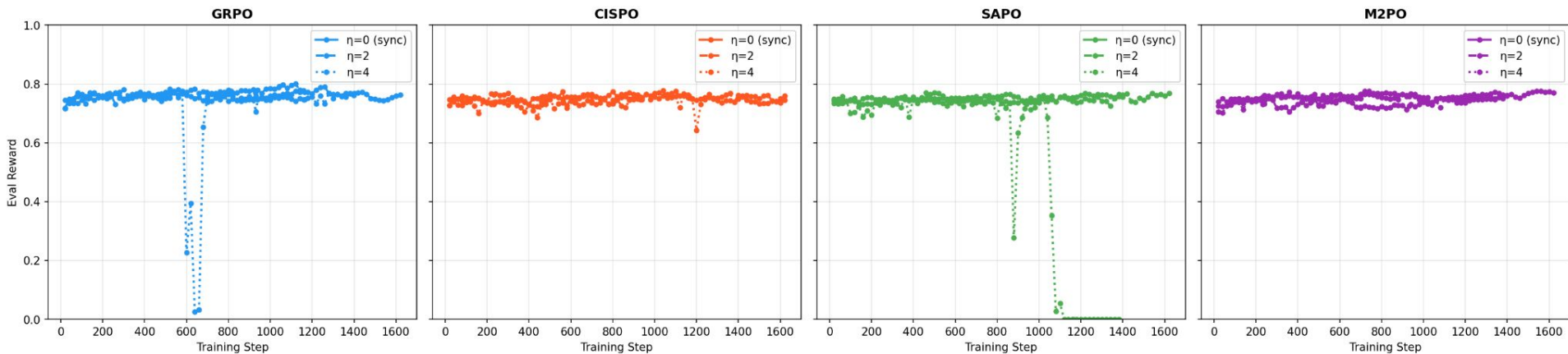
Final Reward Heatmap



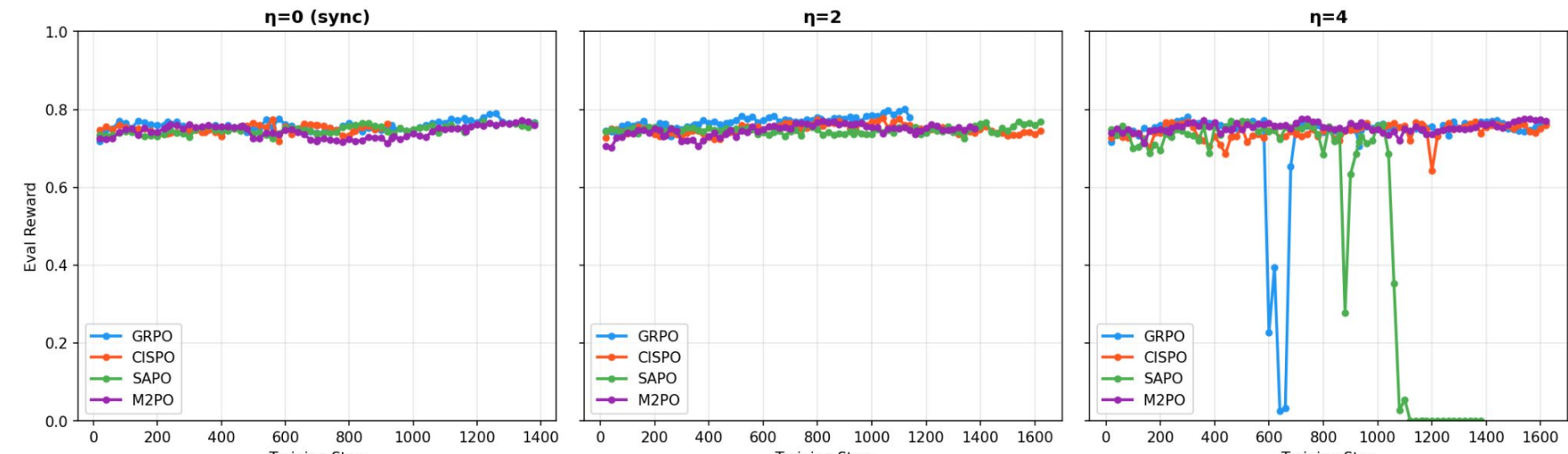
Effect of Staleness on Final Reward



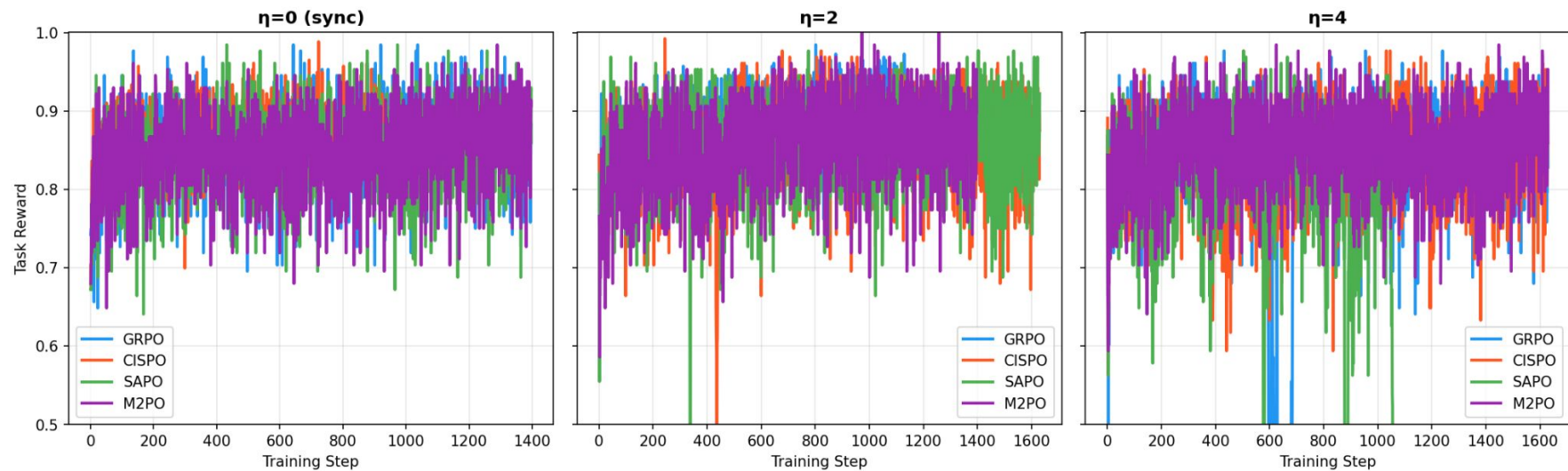
Eval Reward Curves by Algorithm (GRPO vs CISPO vs SAPO vs M2PO)



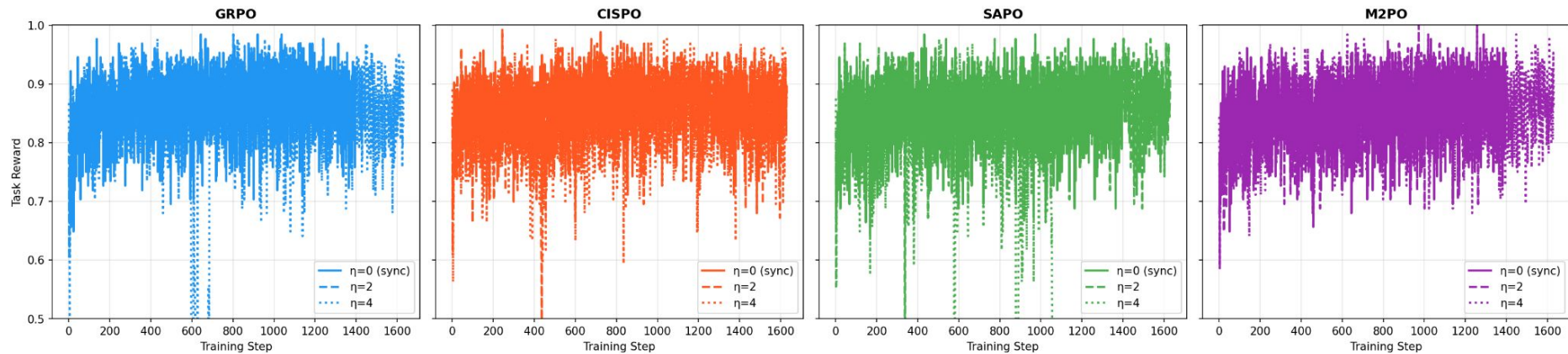
Eval Reward Curves by Staleness Level ($\eta=0, 2, 4$)



Reward Curves by Staleness Level ($\eta=0, 2, 4$)



Reward Curves by Algorithm (GRPO vs CISPO vs SAPO vs M2PO)



Discussion: Algorithm Analysis & Validation



Staleness Hierarchy

- **CISPO:** 89.84% at $\eta=4$ (Best under extreme staleness)
- **M2PO:** 85.94% at $\eta=4$ (Second-best performance)
- **GRPO:** 88.28% $\rightarrow$ 82.81% (Linear decline)
- **SAPO:** Catastrophic failure at $\eta=4$ (Brittle)



Design Validation

- **CISPO:** Conservative importance sampling handles stale gradients.
- **M2PO:** Off-policy corrections maintain training stability.
- **SAPO:** Aggressive optimization breaks at high thresholds.



Practical Implications

High-Staleness: CISPO is the clear winner for extreme conditions.

Reliability: M2PO offers the most consistent across-the-board performance.

Risk: SAPO requires strict management; high reward but high risk.

Limitations & Scope



GSM8K Benchmark

Task Complexity: Qwen-1.5B already solves most problems; advantages remain narrow.

Understatement: Staleness effects are likely understated compared to harder benchmarks.



Model Scale (1.5B)

Distribution: Larger models produce sharper distributions and bigger ratio shifts.

Extrapolation: Results may not directly apply to 7B+ parameter architectures.



Staleness Range ($\eta=0-4$)

Current Scope: Limited to 0-4 weight updates due to compute constraints.

Target: AReal suggests η values of 2-8 are needed for a comprehensive study.



Single Task Domain

Focus: Findings currently limited to mathematical reasoning tasks.

Future Voids: No evidence yet for code generation, dialogue, or RLHF-style tasks.

Conclusion + Future Work

Conclusions

- **Robustness:** CISPO excels in staleness robustness, validating its asynchronous training design.
- **Alignment:** Algorithm selection should align with expected system staleness.
- **Performance:** Conservative methods (CISPO, M2PO) beat aggressive ones under high staleness.
- **Context:** Real-world staleness differs from simulations.

Future Work

- Increase staleness ($\eta = 6, 8$) to find CISPO's limit and confirm M2PO's stability.
- Scale to Qwen-7B with harder benchmarks (MATH500, AIME).
- Implement adaptive staleness scheduling.
- Study convergence over longer training periods.

Impact

Distributed RL: CISPO enables wider adoption in distributed reinforcement learning.

Algorithm Choice: Provides evidence-based choices for scalable RL systems.